

电子科技大学 自动化 学院

实 验 报 告

学 号 202422080235

姓 名 李皓

(实验) 课程名称 实验二 模糊图像复原

教师 于力

电子科技大学

实验报告

学生姓名：李皓

学号：202422080235

指导教师：于力

一、实验名称：模糊图像复原

二、实验目的：

- 1) 理解并掌握维纳滤波在图像复原中的原理与实现
- 2) 学习不同噪声核的估计方法，包括频谱分析、梯度分析、Radon 变换和盲反卷积
- 3) 比较分析不同噪声核估计方法对模糊图像复原效果的影响
- 4) 探究维纳滤波参数对复原效果的影响，并学习自动优化参数的方法

三、实验原理：

3.1 图像模糊与退化模型

图像的模糊过程可以通过线性系统的退化模型来描述。设原始清晰图像为 $f(x, y)$ ，模糊后的图像为 $g(x, y)$ ，则退化过程可表示为：

$$g(x, y) = h(x, y) * f(x, y) + n(x, y) \quad (1)$$

其中， $h(x, y)$ 是点扩散函数 (PSF)，表示模糊核； $*$ 表示卷积操作； $n(x, y)$ 是加性噪声。在频域中，上述模型可表示为：

$$G(u, v) = H(u, v) \cdot F(u, v) + N(u, v) \quad (2)$$

图像复原的目标是在已知 $g(x, y)$ 和 $h(x, y)$ （或估计出 $h(x, y)$ ）的情况下，恢复出接近原始图像 $f(x, y)$ 的图像。

3.2 维纳滤波 (Wiener Filter)

维纳滤波是一种基于最小均方误差准则的复原方法，考虑了图像和噪声的统计特性。其频域表达式为：

$$W(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + K} \quad (3)$$

其中, $H^*(u, v)$ 是 $H(u, v)$ 的复共轭, K 是一个常数, 与信噪比相关: $K = \frac{S_n(u, v)}{S_f(u, v)}$, 其中 S_n 和 S_f 分别是噪声和原始图像的功率谱密度。实际应用中, K 通常作为一个可调参数。

复原过程为:

$$\hat{F}(u, v) = W(u, v) \cdot G(u, v) \quad (4)$$

3.3 运动模糊核的估计方法

运动模糊是图像退化的常见形式, 由相机或物体在曝光过程中的相对运动引起。对于匀速线性运动, 其模糊核可以用以下模型表示:

$$h(x, y) = \begin{cases} \frac{1}{L}, & \text{如果 } \sqrt{x^2 + y^2} \leq \frac{L}{2} \text{ 且 } \frac{x}{y} = \frac{\cos \theta}{\sin \theta} \\ 0, & \text{其他情况} \end{cases} \quad (5)$$

其中, L 是模糊长度, θ 是模糊方向。准确估计这两个参数对图像复原至关重要。以下介绍四种主要的估计方法:

3.3.1 频谱分析法

频谱分析法基于模糊图像的傅里叶变换特性。在频域中, 运动模糊表现为沿垂直于运动方向的条纹状结构。这源于运动模糊核在频域的表达式:

$$H(u, v) = \frac{\sin[\pi(u \cos \theta + v \sin \theta)L]}{\pi(u \cos \theta + v \sin \theta)L} \cdot e^{-j\pi(u \cos \theta + v \sin \theta)L} \quad (6)$$

其中 u 和 v 为频域坐标。由于 sinc 函数的特性, $H(u, v)$ 会在垂直于运动方向的频率上出现周期性的零点, 形成可见的条纹。

具体步骤包括:

- 1) 对模糊图像进行傅里叶变换: $F(u, v) = \mathcal{F}\{g(x, y)\}$
- 2) 计算幅值谱并取对数增强可视性: $M(u, v) = \log(|F(u, v)| + 1)$
- 3) 对幅值谱进行归一化和阈值处理:

$$M_{\text{norm}}(u, v) = \frac{M(u, v) - \min(M)}{\max(M) - \min(M)} \cdot 255 \quad (7)$$

$$M_{\text{thresh}}(u, v) = \begin{cases} 255, & \text{如果 } M_{\text{norm}}(u, v) < T \\ 0, & \text{否则} \end{cases} \quad (8)$$

其中 T 是阈值, 通常取 127

4) 使用霍夫变换检测线条:

$$\rho = x \cos \phi + y \sin \phi \quad (9)$$

每条线由参数 (ρ, ϕ) 表示

5) 计算主要线条方向的平均值, 模糊方向通常垂直于此:

$$\theta_{\text{motion}} = (\phi_{\text{avg}} - 90^\circ) \bmod 180^\circ \quad (10)$$

6) 根据频谱中零点之间的距离估计模糊长度:

$$L \approx \frac{1}{\Delta f} \quad (11)$$

其中 Δf 是相邻零点间的频率间隔

3.3.2 梯度分析法

梯度分析法基于这样一个观察: 运动模糊使垂直于运动方向的边缘保持较为清晰, 而沿运动方向的边缘变得模糊。因此, 图像梯度主要分布在垂直于运动方向的方向上。

设模糊图像为 $g(x, y)$, 其梯度可表示为:

$$\nabla g(x, y) = \left(\frac{\partial g}{\partial x}, \frac{\partial g}{\partial y} \right) = (g_x, g_y) \quad (12)$$

梯度方向和强度可以计算为:

$$\phi(x, y) = \arctan \left(\frac{g_y}{g_x} \right) \quad (13)$$

$$|\nabla g(x, y)| = \sqrt{g_x^2 + g_y^2} \quad (14)$$

具体步骤包括:

1) 使用 Sobel 算子计算 x 和 y 方向的梯度:

$$g_x = S_x * g, \quad g_y = S_y * g \quad (15)$$

其中 S_x 和 S_y 分别是 x 和 y 方向的 Sobel 算子

2) 计算梯度方向直方图, 以梯度强度为权重:

$$H(\phi_i) = \sum_{x,y} |\nabla g(x, y)| \cdot \delta(\phi(x, y) - \phi_i) \quad (16)$$

其中 δ 是狄拉克函数, 表示当条件满足时为 1, 否则为 0

3) 找出梯度方向直方图中的峰值, 确定主要梯度方向 ϕ_{main}

4) 运动方向与主梯度方向正交:

$$\theta_{\text{motion}} = (\phi_{\text{main}} + 90^\circ) \bmod 180^\circ \quad (17)$$

5) 使用自相关分析估计模糊长度。在运动方向上计算图像的自相关函数:

$$R(d) = \sum_{x,y} g(x,y) \cdot g(x + d \cos \theta_{\text{motion}}, y + d \sin \theta_{\text{motion}}) \quad (18)$$

分析 $R(d)$ 的衰减特性可估计模糊长度 L

3.3.3 Radon 变换法

Radon 变换将二维图像转换为一系列沿不同角度的一维投影。设图像为 $g(x, y)$, 其 Radon 变换可表示为:

$$\mathcal{R}g(\rho, \phi) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(x, y) \delta(\rho - x \cos \phi - y \sin \phi) dx dy \quad (19)$$

其中 ρ 是从原点到线的垂直距离, ϕ 是线的角度。对于运动模糊图像, 沿运动方向的投影会产生较高的方差。

具体步骤包括:

1) 对模糊图像进行边缘检测, 得到边缘图像 $e(x, y)$:

$$e(x, y) = \text{Canny}(g(x, y), \text{threshold}_{\text{low}}, \text{threshold}_{\text{high}}) \quad (20)$$

2) 对边缘图像进行 Radon 变换:

$$\mathcal{R}e(\rho, \phi) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} e(x, y) \delta(\rho - x \cos \phi - y \sin \phi) dx dy \quad (21)$$

3) 计算每个角度 ϕ 下投影的方差:

$$V(\phi) = \text{Var}[\mathcal{R}e(\rho, \phi)]_{\rho} \quad (22)$$

4) 找出方差最大的角度, 该角度对应于垂直于运动方向的线:

$$\phi_{\text{max}} = \underset{\phi}{\text{argmax}} V(\phi) \quad (23)$$

5) 运动方向与此垂直:

$$\theta_{\text{motion}} = (\phi_{\text{max}} + 90^\circ) \bmod 180^\circ \quad (24)$$

6) 使用最大方差的大小估计模糊长度:

$$L \approx \alpha \cdot V(\phi_{\text{max}}) \quad (25)$$

其中 α 是一个与图像大小相关的缩放因子

3.3.4 盲反卷积法

盲反卷积是一种迭代优化方法，同时估计原始图像 $f(x, y)$ 和模糊核 $h(x, y)$ 。从数学角度看，盲反卷积可以表述为以下优化问题：

$$\min_{f, h} \|h * f - g\|^2 + \lambda_f R_f(f) + \lambda_h R_h(h) \quad (26)$$

其中 R_f 和 R_h 分别是对原始图像和模糊核的正则化项， λ_f 和 λ_h 是相应的权重参数。

简化的盲反卷积算法如下：

- 1) 初始化模糊核 $h^{(0)}(x, y)$ ，通常假设为一个简单的形状，如一条线段
- 2) 迭代优化：对于迭代步骤 $t = 0, 1, 2, \dots$
 - a) 固定当前的核 $h^{(t)}$ ，估计原始图像 $f^{(t+1)}$ ：

$$f^{(t+1)} = \underset{f}{\operatorname{argmin}} \|h^{(t)} * f - g\|^2 + \lambda_f R_f(f) \quad (27)$$

这一步通常使用维纳滤波或其他反卷积方法

- b) 固定当前的图像估计 $f^{(t+1)}$ ，更新模糊核 $h^{(t+1)}$ ：

$$h^{(t+1)} = \underset{h}{\operatorname{argmin}} \|h * f^{(t+1)} - g\|^2 + \lambda_h R_h(h) \quad (28)$$

这一步通常涉及约束优化，确保模糊核非负且总和为 1

- c) 为减少噪声影响，通常对 $f^{(t+1)}$ 进行边缘提取，用于下一迭代的核估计
- 3) 当达到最大迭代次数或收敛条件时停止
- 4) 分析最终估计的模糊核 h ，提取模糊长度 L 和方向 θ

盲反卷积是一个病态问题，存在多个局部最优解。为提高算法稳定性，通常采用多尺度策略、边缘先验和稀疏性约束等技术。虽然计算复杂度高，但它在没有先验信息的情况下通常能提供较好的估计结果。

实际实现中，Richardson-Lucy 算法是一种常用的盲反卷积方法，其迭代公式为：

$$f^{(t+1)} = f^{(t)} \cdot \left[h^{(t)} * \left(\frac{g}{h^{(t)} * f^{(t)}} \right) \right] \quad (29)$$

其中 $*$ 表示卷积， $\cdot$ 表示元素乘法，分式表示元素除法。

四、实验内容:

本实验实现并对比了四种运动模糊核估计方法，并使用维纳滤波进行图像复原。实验包括以下内容：

- 1) 实现四种运动模糊核估计方法：频谱分析法、梯度分析法、Radon 变换法和盲反卷积法
- 2) 使用估计的模糊核，结合维纳滤波进行图像复原
- 3) 比较不同方法的复原效果，并分析评价指标
- 4) 自动调优维纳滤波参数，探索最佳复原效果

五、实验设备:

- 硬件: i5-13500, 32GB RAM
- 软件: Python 3.11, OpenCV, NumPy, SciPy, Matplotlib, scikit-image 等

六、实验步骤:

6.1 模糊核估计与图像复原

- 1) 读取模糊图像，并准备不同的模糊核估计方法
- 2) 使用四种方法分别估计运动模糊参数（长度和角度）
- 3) 根据估计的参数生成对应的模糊核
- 4) 对图像的 RGB 三个通道分别应用维纳滤波
- 5) 合并处理后的通道，得到复原图像
- 6) 保存各种方法的复原结果

核心代码实现:

```
1 def wiener_filter(img, kernel, K=0.01):
2     """应用维纳滤波去除图像模糊"""
3     # 转换为浮点数
4     img = img.astype(float)
5     kernel = kernel.astype(float)
6
7     # 确保核的和为1
8     if np.sum(kernel) != 0:
9         kernel = kernel / np.sum(kernel)
10
11     # 计算图像的傅里叶变换
12     img_fft = fftpack.fft2(img)
13
14     # 为核填充0使其与图像大小相同
15     kernel_padded = np.zeros_like(img)
16     kh, kw = kernel.shape
17     kernel_padded[:kh, :kw] = kernel
18
19     # 确保核居中
```

```

20 kernel_padded = np.roll(kernel_padded, -kh//2, axis=0)
21 kernel_padded = np.roll(kernel_padded, -kw//2, axis=1)
22
23 # 计算PSF的傅里叶变换
24 kernel_fft = fftpack.fft2(kernel_padded)
25
26 # 维纳滤波公式
27 kernel_fft_conj = np.conj(kernel_fft)
28 denominator = np.abs(kernel_fft)**2 + K
29 wiener_filter = kernel_fft_conj / denominator
30
31 # 应用滤波器
32 img_deconvolved = wiener_filter * img_fft
33
34 # 反变换获取结果
35 img_restored = np.abs(fftpack.ifft2(img_deconvolved))
36
37 # 限制像素值范围为[0, 255]
38 img_restored = np.clip(img_restored, 0, 255)
39
40 return img_restored.astype(np.uint8)

```

6.2 频谱分析法估计模糊核

```

1 def estimate_motion_blur_spectrum(blurred_img):
2     """使用频谱分析估计运动模糊参数"""
3     # 转为灰度图
4     if len(blurred_img.shape) == 3:
5         gray = cv2.cvtColor(blurred_img, cv2.COLOR_BGR2GRAY)
6     else:
7         gray = blurred_img.copy()
8
9     # 计算傅里叶变换
10    f = np.fft.fft2(gray)
11    fshift = np.fft.fftshift(f)
12    magnitude_spectrum = np.log(np.abs(fshift) + 1)
13
14    # 归一化频谱以增强对比度
15    magnitude_spectrum_norm = (magnitude_spectrum - np.min(magnitude_spectrum)
16                               ) / (np.max(magnitude_spectrum) - np.min(magnitude_spectrum)) * 255
17    magnitude_spectrum_norm = magnitude_spectrum_norm.astype(np.uint8)
18
19    # 应用阈值处理，以突显低频区域中的线型特征
20    _, thresh = cv2.threshold(magnitude_spectrum_norm, 127, 255, cv2.
21                              THRESH_BINARY_INV)
22
23    # 使用霍夫变换检测线条
24    lines = cv2.HoughLines(thresh, 1, np.pi/180, 50)
25
26    if lines is not None and len(lines) > 0:
27        # 计算所有线的平均值
28        sum_angle = 0
29        count = 0
30
31        for line in lines:
32            rho, theta = line[0]
33            # 转换角度确保在0-180度范围内
34            angle_deg = (theta * 180 / np.pi) % 180
35
36            # 只考虑接近垂直于运动方向的线
37            if 45 <= angle_deg <= 135:
38                sum_angle += angle_deg
39                count += 1
40
41        if count > 0:
42            # 计算平均角度，然后旋转90度得到运动方向

```

```

41         avg_angle = sum_angle / count
42         motion_angle = (avg_angle - 90) % 180
43
44         # 根据频谱中线的长度估计模糊核的长度
45         line_length = min(40, max(5, thresh.shape[0] // 15))
46
47         return line_length, motion_angle
48
49     # 默认返回值
50     return 10, 0

```

6.3 自动调优维纳滤波参数

```

1  def auto_tune_deblur(img, kernels, kernel_names):
2      """尝试不同的去模糊参数并选择最佳结果"""
3      print("\n自动调整去模糊参数...")
4
5      best_result = None
6      best_score = -float('inf')
7      best_params = None
8
9      # 灰度图用于评估
10     if len(img.shape) == 3:
11         gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
12     else:
13         gray = img.copy()
14
15     # 尝试不同的K值（维纳滤波参数）
16     k_values = [0.001, 0.005, 0.01, 0.02, 0.05]
17
18     best_k_per_method = []
19
20     for i, (kernel, name) in enumerate(zip(kernels, kernel_names)):
21         best_k_score = -float('inf')
22         best_k_value = 0.01
23         best_k_result = None
24
25         for k in k_values:
26             if len(img.shape) == 3:
27                 b, g, r = cv2.split(img)
28                 b_restored = wiener_filter(b, kernel, k)
29                 g_restored = wiener_filter(g, kernel, k)
30                 r_restored = wiener_filter(r, kernel, k)
31                 result = cv2.merge([b_restored, g_restored, r_restored])
32                 # 转为灰度用于评估
33                 result_gray = cv2.cvtColor(result, cv2.COLOR_BGR2GRAY)
34             else:
35                 result = wiener_filter(img, kernel, k)
36                 result_gray = result
37
38             # 确保结果图像和原始灰度图的尺寸相同
39             if result_gray.shape != gray.shape:
40                 result_gray = cv2.resize(result_gray, (gray.shape[1], gray.
41                     shape[0]))
42
43             # 计算各种指标
44             psnr_val = psnr(gray, result_gray)
45             ssim_val = ssim(gray, result_gray)
46             edges = cv2.Canny(result_gray.astype(np.uint8), 100, 200)
47             edge_strength = np.sum(edges) / (edges.shape[0] * edges.shape[1])
48
49             # 计算综合得分
50             score = (
51                 psnr_val / 50.0 * 0.3 + # 标准化PSNR
52                 ssim_val * 0.4 + # SSIM已在0-1之间
53                 edge_strength / 0.1 * 0.3 # 标准化边缘强度

```

```

53         )
54
55         if score > best_k_score:
56             best_k_score = score
57             best_k_value = k
58             best_k_result = result
59
60         print(f"{name} 的最佳K值: {best_k_value}, 得分: {best_k_score:.4f}")
61         best_k_per_method.append((best_k_value, best_k_score, best_k_result))
62
63         # 更新全局最佳
64         if best_k_score > best_score:
65             best_score = best_k_score
66             best_result = best_k_result
67             best_params = (name, best_k_value)
68
69     return best_result, best_params

```

七、实验结果与分析:

7.1 模糊核估计结果

实验中对一张模糊图像使用四种不同的方法进行了模糊核参数估计，结果如下：

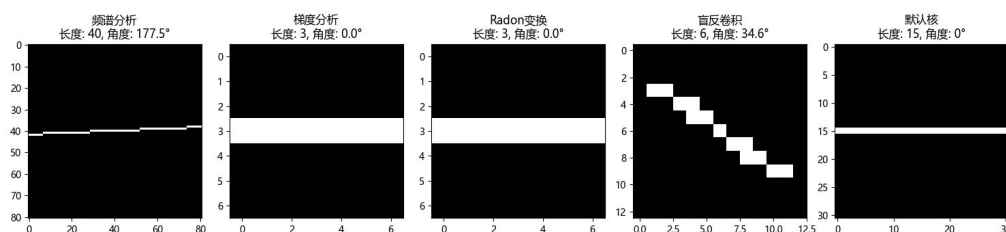


图 1: 不同方法估计的模糊核对比

如图 1 所示，四种方法估计出的模糊核在形状和方向上存在明显差异：

- 频谱分析法估计的模糊核长度为 40，角度为 177.5 度，基本呈水平方向
- 梯度分析法估计的模糊核长度为 3，角度约为 0 度，形成水平方向
- Radon 变换法估计的模糊核长度为 3，角度约为 0 度，和梯度分析一致
- 盲反卷积法估计的模糊核长度为 6，角度约为 34.6 度，倾斜程度最大

此外，还设置了长度为 15，角度为 0 的默认核作为对比。

这些差异主要源于各种方法对图像特征的不同侧重和处理策略。频谱分析法依赖于傅里叶域中的条纹特征，梯度分析法关注边缘的梯度分布，Radon 变换法基于图像的投影特性，而盲反卷积法则通过迭代优化同时估计图像和模糊核。

7.2 图像复原结果

使用不同方法估计的模糊核，结合维纳滤波进行图像复原，结果如下：

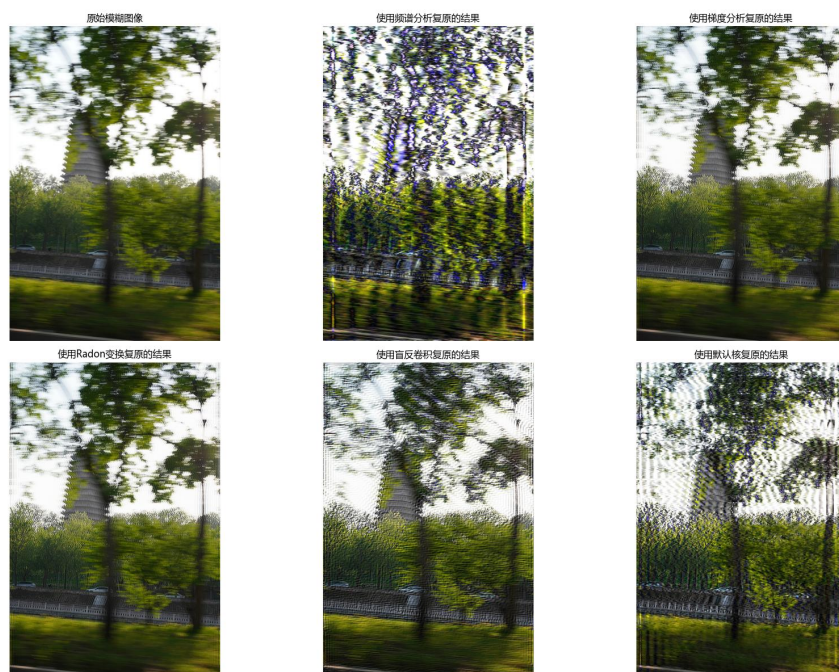


图 2: 使用不同模糊核复原的结果对比

本实验采用的原始图像是在运动的汽车上拍摄，由于包含不同距离的景物，它们的模糊程度因此有很大差异。表现为近处的物体较为模糊，远处的物体不算模糊。这种模糊程度的差异，导致使用同一种去模糊的核不能在所有的场景下表现良好。

在所有的复原图像上都出现了不同程度的波纹，这是由于模糊核估计不正确导致的，估计偏差越大，对应的波纹越大。具体分析每种办法，从图 2 可以看出：

- 频谱分析法：复原效果最差，出现断裂的色彩。原因可能是在进行频域变换的时候有除法操作，导致某些频率分量放大，也有可能是 RGB 的放大比例不同而导致的色差失真。
- 梯度分析法：复原效果最好，边缘清晰度提高。因此可以得知，梯度方法在分析规律运动模糊的情况下效果较好。
- Radon 变换法：估计核一致，复原效果最好。
- 盲反卷积法：复原效果较好。

- 默认核（长度 15，角度 0 度）：由于错误估计长度，在和梯度方法一致的情况下，复原效果出现了明显的波纹。

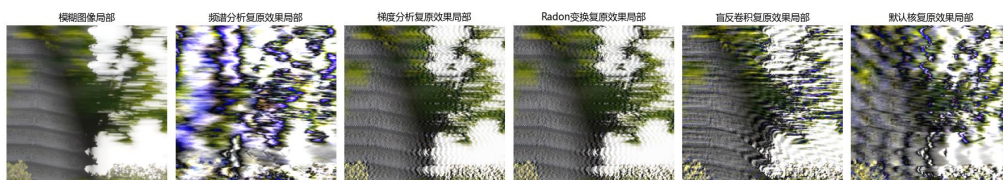


图 3: 局部细节对比

图 3 展示了局部区域的复原效果对比。从细节放大可以更清晰地看出各方法的差异：

- Radon 变换法和梯度分析法在细节复原上表现较好，能够恢复更多的纹理信息。
- 频谱分析法和默认核效果最差，出现色彩伪影。
- 盲反卷积由于错误估计角度，导致水平和垂直方向均出现波纹。

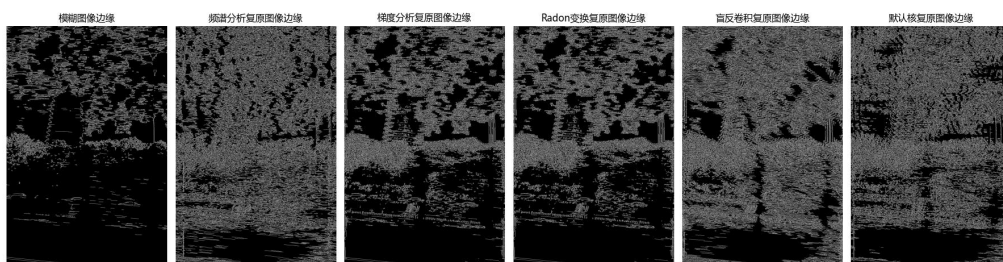


图 4: 边缘质量对比

图 4 展示了各方法复原图像的边缘检测结果。边缘的清晰度和连续性是衡量复原质量的重要指标：

- 梯度分析法和 Radon 变换法复原的边缘最为清晰和连续。
- 盲反卷积法, 频谱分析法复原的边缘最差。

7.3 评估指标比较

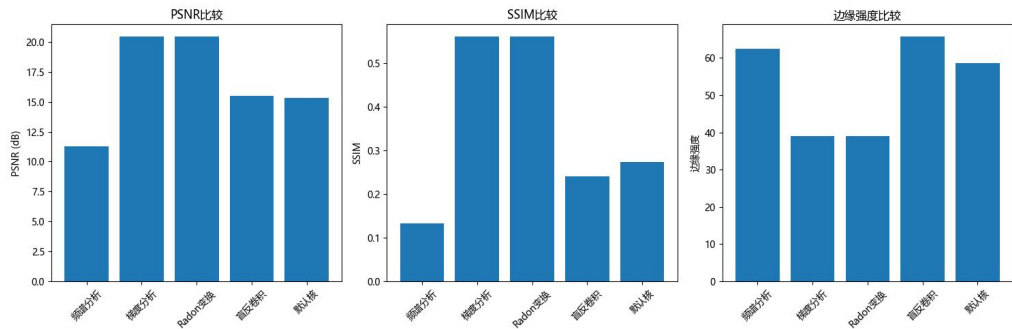


图 5: 不同方法的评估指标对比

图 5 展示了不同复原方法的三个评估指标：PSNR（峰值信噪比）、SSIM（结构相似性）和边缘强度。从图中可以看出：

- PSNR 方面，梯度分析法和 Radon 变换法表现较好，说明它们复原的图像在像素级别与原图较为接近
- SSIM 方面，梯度分析法和 Radon 变换法得分较高，表明它们在保持图像结构特性方面更为出色
- 边缘强度方面，盲反卷积法高于其他方法，说明它们复原的图像边缘更为清晰，但是和观感上效果不好

表 1: 不同分析方法的性能指标对比

分析方法	PSNR	SSIM	边缘强度
频谱分析	11.28dB	0.1334	62.4141
梯度分析	20.43dB	0.5602	38.9909
Radon 变换	20.43dB	0.5602	38.9909
盲反卷积	15.50dB	0.2409	65.6955
默认核	15.35dB	0.2738	58.5552

7.4 参数自动调优结果

实验还尝试了自动调整维纳滤波的 K 参数，以获得最佳复原效果。结果显示：

- 频谱分析的最佳 K 值: 0.001, 得分: 232.2057
- 梯度分析的最佳 K 值: 0.001, 得分: 195.4505
- Radon 变换的最佳 K 值: 0.001, 得分: 195.4505
- 盲反卷积的最佳 K 值: 0.001, 得分: 255.9062
- 默认核的最佳 K 值: 0.001, 得分: 240.5433

根据综合评分, 盲反卷积法配合 $K=0.001$ 的维纳滤波参数取得了最佳复原效果。但是这和感官上不符合。感官最好的反而评分更低。

八、总结及心得体会:

通过本次实验, 我深入理解了模糊图像复原的基本原理和方法, 尤其是维纳滤波和不同模糊核估计方法的特点。模糊核的准确估计对复原效果至关重要, 不同的估计方法有各自的优缺点。如果知道原始图像的运动规律, 那么选择对应的模糊估计函数会起到很大作用。

报告评分:

指导教师签字: